PROFILE

핵심 역량

공공 판례 4 만 건을 수집 · 모델링 · 적재하고 , 실서비스로 운영한 데이터 엔지니어

역량	수행 내역	성과
① 데이터 수집 ·ETL	· 공공 REST API 2 단계 (목록→본문) 수집 자동화 · 체크포인트 기반 대량 데이터 수집 구조 설계	**판례 수집 11 배 확대** 판례 3,503 건 → 41,438 건
② 데이터 품질 · 정합성	· 판결문 파싱 , 법원명 표준화 , 중복 판례 제거 · 설계 문서와 운영 DB 스키마 전수 대조 · 정정	**데이터 정합성 확보** 법원명 224→73 종 표준화 , 중복 판례 66 건 제거
③ 데이터 모델링 ·DB 설계	· 요구사항 정의부터 논리 · 물리 ERD, FK· 인덱스 설계 · SQLite → PostgreSQL 이관 수행	**표준 기반 DB 설계** 표준 용어 97 개 · 테이블 12 개 설계
④ 개발환경 구성 · 운영	· AWS EC2(Linux) 서버 구축 및 서비스 배포 · 성능 저하 · 메모리 장애 원인 추적 및 해결	**검색 속도 · 서버 안정성 개선** 검색 최대 60 초→ 0.68 초 · 메모리 3.2GB→687MB
차별화 역량		
⑤ 비정형 데이터 처리	· 판례 72 만 문단 임베딩 , 벡터 DB 분할 적재 · 키워드 + 벡터 하이브리드 검색 , 개인화 RAG 연동	**의미 기반 검색 서비스 확장** 판례 72 만 문단 적재 · 유사 판례 검색 고도화

자격 · 교육

DAsP 데이터아키텍처 준전문가 취득
SQLD · OCP 2026.11 응시 예정
오라클 DB 전문가 과정 수강 중
아이티윌 · 26.06~26.11 · 튜닝 · 백업 / 복구
오라클 교육 학습 일지 ↗

평가 · 수상

2026 한이음 드림업 2 개 과제 (2026.4~10)
중간평가 상위 30%

근거 프로젝트

LawSight ↗
팀장 · Data·DB·Infra 총괄 | 4 인

My Task Pilot ↗
Data/AI 담당 | 3 인

영화 예매 플랫폼 ↗
단독 · ERD 설계 ·DB 구축

학력

수원대 컴퓨터소프트웨어학과 · 2027.02 졸업예정

ROLE MAP

프로젝트별 수행 역할

모든 프로젝트에서 데이터 파트 주도

프로젝트 · 역할	서비스 구성별 수행 범위			프로젝트 단계별 수행 범위		
	데이터	기능	서비스	분석	설계	개발
LawSight 팀장 · 4 인 · 2026.04~10 AI 판례 분석 · 소송전략	● 주도 수집 · 정제 ·DB· 벡터 DB	◐ 협업 검색 연동 · 병목 개선	● 주도 EC2 배포 · 운영	● 주도 요구사항 35 건 총괄	◐ 협업 테이블 / 기능 / 화면 설계	◐ 협업 파이프라인 / 기능 / 화면
MyTaskPilot Data/AI · 3 인 · 2026.04~10 AI Agent 업무 자동화	● 주도 저장 · 벡터 ·DB 이관	◐ 협업 RAG 개인화 기획 · 검증	◐ 협업 제출 문서 총괄	◐ 협업 요구사항정의	◐ 협업 테이블 / 기능 / 화면 설계	◐ 협업 기능 구현 / 테스트
영화 예매 플랫폼 단독 · 2026.01~02 표준 기반 데이터 모델링	● 주도 논리 · 물리 ERD	● 주도 저장 및 화면쿼리	● 주도 웹 화면 구현	● 주도 요구사항 정의서	● 주도 테이블 / 기능 / 화면 설계	● 주도 Node.js · PostgreSQL

기타 개인 프로젝트

맞춤형 식단 관리 웹 2025.07~08 · 단독 · 공공데이터 수집 , 칼로리 · 영양 집계 , 차트 시각화 화면 구현 (Spring Boot·MyBatis)
신용 리스크 예측 2025.10~11 · 단독 · 결측 · 불균형 보정 , 10 개 AI/ML 모델 비교 · 선정 , 정확도 AUROC 0.8585

● 주도 = 본인이 주도한 영역 ◐ 협업 = 팀원과 협업한 영역

1. LawSight > AI 판례 분석 및 소송전략 시스템

변호사가 자연어로 사건을 입력하면 유사 판례를 찾고 소송 전략 리포트를 생성하는 AI 리서치 보조 서비스

구분	본인 역할	기술 스택	서비스
2026 한이음 드림업 (과기정통부) · Team 404 · 4 인 · 2026.04 ~ 10	팀장 (PM) 겸 Data · DB · 배포 인프라 총괄	PostgreSQL · ChromaDB · KoSBERT · BM25+RRF · FastAPI · GPT-4o · AWS EC2	www.lawsight.co.kr

01 해결과제 무엇이 어려웠나	02 액션 내가 한 일	03 결과 before → after
상황 판례 리서치는 수작업, 원천 판결문은 형식이 제각각인 비정형 텍스트		
1 **수집** · 중단 시 처음부터 재수집해야 하는 대량 수집 구조	**2 단계 REST 수집 자동화** · 목록 → 본문 자동 수집, 중단 지점부터 재개	**판례 데이터 11 배 확보** 3,503 건 → 41,438 건
2 **정제** · 같은 법원이 224 가지 이름으로 기록, 중복 판례 혼재	**표준화 · 정합성 검증 스크립트** · 법원명 정규화 · 중복 제거 후 정합성 자동 검증	**법원명 표준화** 224 종 → 73 종 (약 67%↓) · 중복 66 건 제거
3 **저장** · 정형 · 벡터 데이터 대량 적재 중 서버 강제 종료 반복	**정형 + 벡터 저장소 분리 설계** · PostgreSQL 11 개 테이블 + ChromaDB 분할 적재	**판례 72 만 문단 적재 완료** 적재 실패 → 분할 저장으로 해결
4 **운영** · 요청마다 인덱스 재로딩 → 검색 지연 · 메모리 부족	**원인 추적 · 서버 구성** · 로그로 원인 추적 → 인덱스 1 회 로딩 전환 (팀 협업)	**검색 속도 개선** 20~60 초 → 0.68 초 / **메모리 79% 감소** 3.2GB → 687MB

실서비스 운영 (EC2 · nginx · HTTPS) · **한이음 중간평가 상위 30%** **배운 점** 검색 품질은 모델보다 데이터 품질과 지속 수집 파이프라인이 좌우함을 확인

1. LawSight > 데이터 파이프라인 · ERD · 운영 구성

수집 ~ 벡터 인덱스까지 데이터 계층을 주도해 구축하고, EC2 단일 서버로 실서비스 운영

데이터 파이프라인

01 수집	02 파싱	03 정제 · 검증	04 적재	05 임베딩
2 단계 REST 수집 체크포인트 재개	**CASE_ / JUDGMENT** 1:1 분리 적재	**법원명 정규화 규칙** 적재 후 검증 자동 실행	**JSONB 가변 항목** GIN 인덱스 2 개	**KoSBERT 청크** 2 개 컬렉션 분할 적재

운영 구성 · 장애 대응

AWS EC2 t3.medium · 단일 서버 · swap 4GB

nginx HTTPS → FastAPI 검색 API

- PostgreSQL 11 테이블
- ChromaDB 2 개 컬렉션
- BM25 383MB 기동 시 1 회 로딩

테이블 구성도 (ERD)

PostgreSQL 11 테이블 · 인덱스 14 개 · FK ON DELETE CASCADE

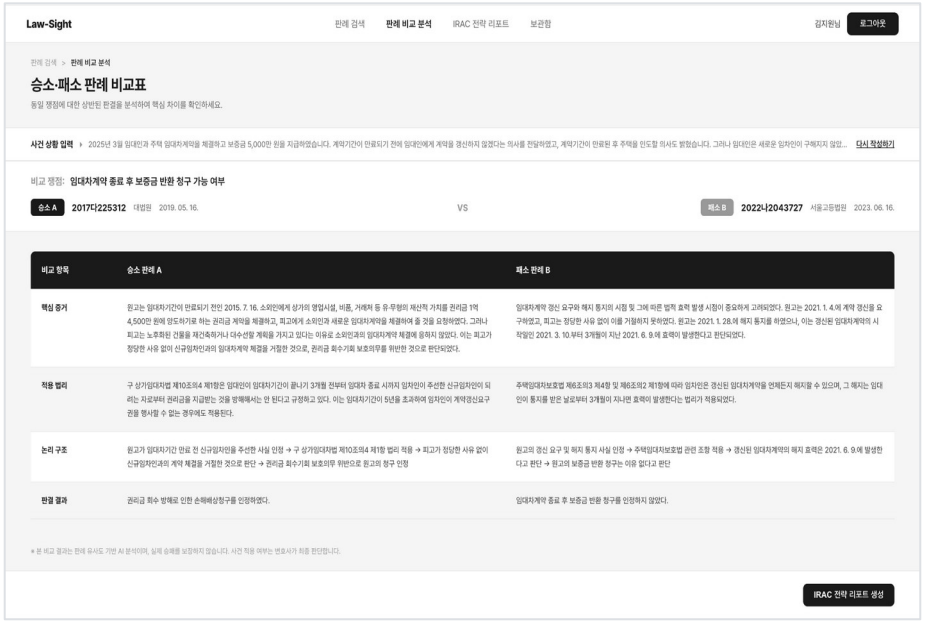


1 CASE_ : JUDGMENT 1:1 분리
판례 메타와 판결 본문을 분리 적재 , 1:1· 중복 · 누락 자동 검증

2 PostgreSQL ↔ ChromaDB 연결
CHROMA_ID 로 벡터 검색 결과를 원래 판례로 역추적

3 JSONB + GIN 인덱스
쟁점 등 가변 항목은 JSONB 로 보존 , GIN 인덱스로 검색

서비스 연결 승소 / 패소 비교표



TB_WIN_LOSS_COMPARISON 의 승소 · 패소 비교 결과가 이 화면에 표시
→ 수집 · 정제한 판례가 서비스에 도달하는 지점

2. MyTaskPilot > AI Agent 업무 자동화 서비스

텍스트 · 음성 (STT) · 이미지 (OCR) 입력을 분석해 일정 · 이메일 · 할 일을 자동 등록하는 개인 업무 비서 서비스

구분	본인 역할	기술 스택	서비스
2026 한이음 드림업 + 졸업 프로젝트 · Team404 · 3 인 · 2026.04 ~ 10	Data / AI : DB 설계 · 관리 , 벡터 스키마 · RAG 연동	PostgreSQL + pgvector · Whisper · GPT-4o-mini Vision · text-embedding-3-small · AWS EC2	www.mytaskpilot.kr

01 해결과제 무엇이 어려웠나	02 액션 내가 한 일	03 결과 before → after
상황 입력은 음성 · 이미지 · 자유 텍스트로 제각각 , 저장소는 로컬 DB 사용하여 안정성 저하		
1 **정형화** · 형식이 제각각인 입력이라 그대로는 저장 · 조회 불가	**정형 저장 스키마 설계 · 연동** · 일정 · 할 일 · 메일 저장 테이블과 실행 이력 테이블 설계	**입력 3 종 , 정형 데이터로 통합** 일정 · 할 일 · 메일로 자동 정리 , 사용자 승인 후 등록
2 **전환 및 이관** · 로컬 SQLite 로는 다중 사용자 · 벡터 검색 · 상시 운영 불가	**PostgreSQL + pgvector 이관** · EC2 에 PostgreSQL + pgvector 설치 , 6 개 테이블 이관	**상시 운영 환경 구축** 로컬 SQLite → AWS EC2 PostgreSQL 이관
3 **개인화** · 과거 이력 저장 구조가 없어 일률적인 응답	**벡터 스키마 설계 · 검색 연동** · 이력 임베딩 저장 · 유사도 검색 연동 (RAG 검증은 팀 협업)	**유사도 기반 개인화 검색 구현** 실제 시나리오 226 건 검증
4 **신뢰성** · AI 오실행 위험 , 설계 문서와 운영 DB 불일치	**운영 DB 전수 대조 · 권한 점검** · 테이블 전수 대조 · 문서 정정 , 권한 취약점 발견 · 공유	**설계 ↔ 운영 DB 일치** 6 개 테이블 전수 대조 후 문서 정정 / **보안 취약점 개선** 다른 사람 정보 접근 차단

실서비스 운영 (EC2 · PostgreSQL + pgvector) · **한이음 중간평가 상위 30%** **배운 점** 정형 + 비정형 데이터 연계 시 활용성 · 사용성이 함께 높아짐을 확인

2. MyTaskPilot > 데이터 파이프라인 · 데이터 모델 · 정합성 검증

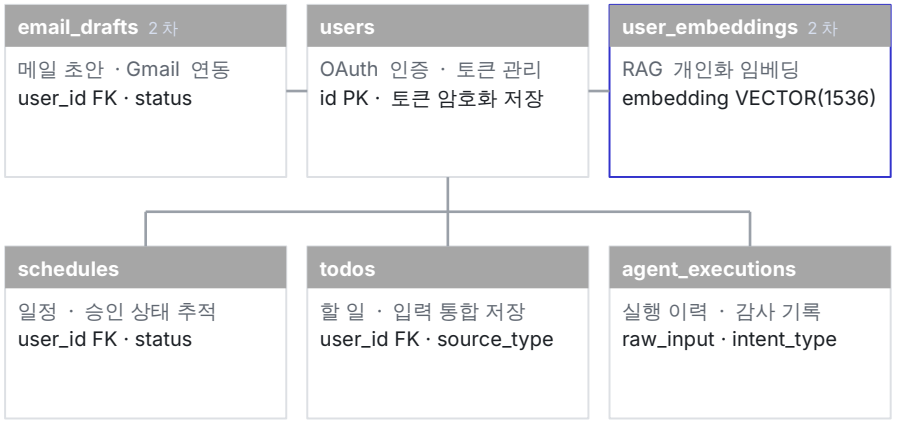
음성·이미지·텍스트를 정형 레코드와 벡터로 저장하고, 운영 DB 기준으로 정합성 검증

비정형 데이터 파이프라인

입력 수집	**텍스트 · 음성 · 이미지 — 형식이 제각각인 비정형 입력** 채팅·음성 녹음·캡처·문서 사진을 하나의 채널로 수용
텍스트화	**Whisper STT · GPT-4o-mini Vision OCR** 음성·이미지를 텍스트로 변환, 샘플로 정확도 검증
구조화	**의도 분류 → 필드 추출 → 사용자 승인 (Function Calling)** 일정·할 일·메일 의도와 필드를 JSON으로 추출
정형 저장	**PostgreSQL 6 테이블 (users 중심)** 일정·할 일·메일 초안 저장, 실행 이력 기록
벡터화 · RAG	**text-embedding-3-small (1536 차원) → pgvector** 사용자 이력 임베딩 저장, 유사 이력 top-3 를 응답에 반영

■ 본인 주도 (저장 · 벡터 스키마) ■ 기획 · 검증 참여 / 팀원 구현 · 본인 연동

데이터 모델



6 개 테이블 · 전 테이블 user_id FK → users

12 → 6 컬럼 정정 (Case 1)

DB 정합성 · 보안 검증

Case 1 — **ERD ↔ 구현 불일치, DB 스키마 전수 검증**

- **문제** 설계 12 vs 구현 6 컬럼, 일부 테이블 미구현
- **원인** 백엔드 변경 사항이 DB 설계 담당에게 미공유
- **조치** 전수대조 → ERD·정의서 수정, '변경 즉시 문서화' 합의

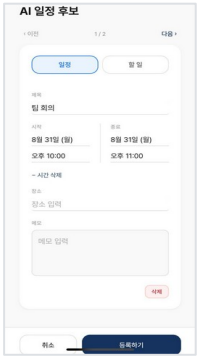
설계 ↔ 운영 DB 테이블 일치

Case 2 — **타인 정보 접근 취약점 발견**

- **문제** 클라이언트가 보낸 user_id 를 서버가 그대로 신뢰
- **원인** 타 사용자 데이터 접근 가능
- **조치** 팀 공유 (본인) → JWT 식별·403 차단 (팀 적용)

서버 접근 보안 개선

서비스 연결 비정형 입력 → 승인 → 정형 레코드



① 일정 등록



② 메일 초안

① **일정 후보 확인 · 수정**
승인 시 **schedules** 저장·상태 추적

② **메일 초안 검토 → Gmail 전송**
email_drafts 에 초안 저장·상태 관리

※ 이력은 **agent_executions** 에 기록

PROJECT 03 · 영화 예매 플랫폼 · 개인

3. 영화 예매 플랫폼 > 표준 기반 데이터 모델링

데이터모델링 교육 직후, 요구사항 정의부터 논리 · 물리 ERD, DB 구축, 예약 기능 구현까지 단독 수행

구분	본인 역할	기술 스택	서비스
3 학년 겨울방학 개인 프로젝트 · 2026.01 ~ 02 (2 개월)	요구사항 · 모델링 · DB 구축 · 구현	DA# Modeler · PostgreSQL · Node.js	movie-booking-platform-ui.vercel.app

01 해결과제 무엇이 어려웠나	02 액션 내가 한 일	03 결과 만든 것
상황 영화 · 상영 · 좌석 · 예매 · 결제가 얽힌 관계형 도메인 설계 필요		
1 **표준화** · 같은 의미가 테이블마다 다른 이름 · 타입으로 생길 위험	**표준 사전 선행 정의** · 도메인 → 단어 → 용어 순 표준 정의 , 상태값은 코드 도메인화	**표준 사전 구축** 도메인 20 개 · 단어 77 개 · 용어 97 개
2 **모델링** · 회원 · 영화 · 상영관 · 좌석 · 예매 · 결제 간 관계 정의 필요	**논리 · 물리 ERD** (DA# Modeler 툴 활용) · 논리 → 물리 ERD 전환 , 테이블정의서 작성	**테이블 12 개 연결 설계** 회원 · 영화 · 좌석 · 예매 · 결제 데이터가 서로 어긋나지 않게 연결
3 **무결성 · 성능** · 잘못된 상태값 입력 시 예약 정합성 훼손	**FK · CHECK · 인덱스 설계** · 예매 · 결제 흐름 FK 연결 , 상태값 CHECK· 조회 인덱스	**오류 차단 · 빠른 조회 설계** CHECK 제약 16 개 · 인덱스 21 개
4 **구현** · 설계 모델의 실제 기능으로 정상 동작 여부 검증 필요	**SQL · API · 화면 구현** · JOIN 기반 예매 · 좌석 쿼리를 API 로 연결 ※ 화면 개발 : AI 코드 도구 활용	**설계 → 예매 기능 구현 완주** 홈 · 예매 · 상영시간표 · 예매내역 · 고객센터 화면 연동

배운 점 표준화 · 무결성 제약을 설계 단계에 반영해야 서비스 품질이 높아짐을 확인 → LawSight 설계 (ERD v1 → v2) 에 적용

3. 영화 예매 플랫폼 > 구현 절차 · ERD · 표준 · 화면

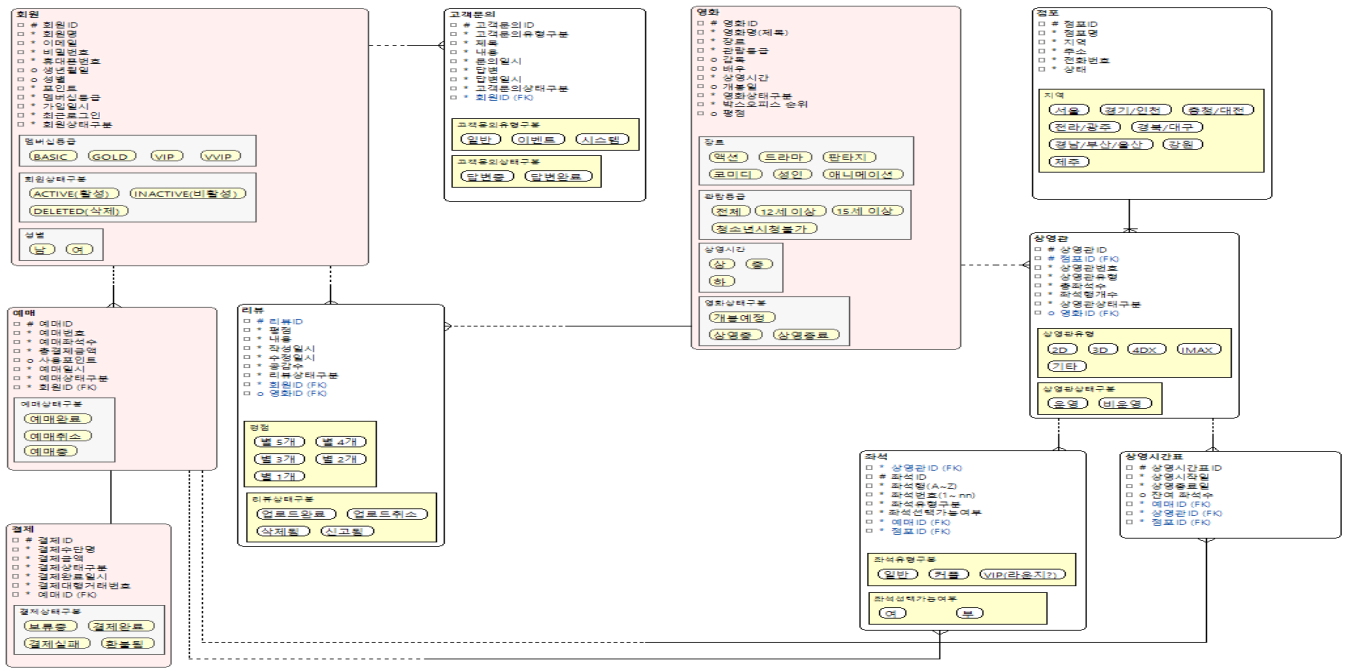
요구사항부터 웹 화면까지 8 단계로 구현하고, 표준 사전 기반으로 12 테이블 설계

구현 절차

요구사항 정의 → 표준 용어 · 도메인 → 논리 ERD → 물리 ERD → PostgreSQL DDL → Express API → 예약 · 좌석 쿼리 → 웹화면

DA# Modeler 로 ERD 설계 → PostgreSQL 구축 → 예약·좌석 쿼리 API 연결 (화면은 AI 코딩 도구 활용)

ERD 테이블 12 개 · FK 12 개 · 인덱스 21 개 · CHECK 16 건 *(DA# Modeler)*



표준 사전 도메인 → 단어 → 용어

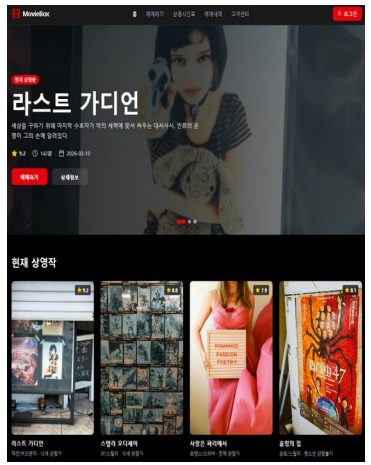
20 개	77 개	97 개
도메인	단어	용어

도메인	약어	데이터 타입
ID	ID	INT
코드 20	CD	VARCHAR(20)
금액	AMT	INT
일시	DTM	TIMESTAMP
점수	SCR	DECIMAL(3,1)

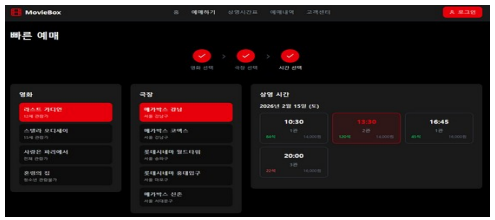
용어 조합 예 회원 + 아이디 → MBR_ID (ID)
멤버십 + 등급 → MBRSHP_LVL (코드 20)

웹 화면 구현

홈



예매



좌석 선택

